

Fibonacci

斐波那契数列

斐波那契数列（The Fibonacci sequence, [OEIS A000045](#)）的定义如下：

该数列的前几项如下：

卢卡斯数列

卢卡斯数列（The Lucas sequence, [OEIS A000032](#)）的定义如下：

该数列的前几项如下：

研究斐波那契数列，很多时候需要借助卢卡斯数列为工具。

斐波那契数列通项公式

第 n 个斐波那契数可以在 $O(n)$ 的时间内使用递推公式计算。但我们仍有更快速的方法计算。

解析解

解析解即公式解。我们有斐波那契数列的通项公式（Binet's Formula）：

这个公式可以很容易地用归纳法证明，当然也可以通过生成函数的概念推导，或者解一个方程得到。

当然你可能发现，这个公式分子的第二项总是小于 1，并且它以指数级的速度减小。因此我们可以把这个公式写成

这里的中括号表示取离它最近的整数。

这两个公式在计算的时候要求极高的精确度，因此在实践中很少用到。但是请不要忽视！结合模意义下二次剩余和逆元的概念，在 OI 中使用这个公式仍是有用的。

卢卡斯数列通项公式

我们有卢卡斯数列的通项公式：

与斐波那契数列非常相似。事实上有：

也就是说， u_n 和 u_{n+1} 恰好构成 $x^2 - 1$ 二项式展开再合并同类项后的分子系数。也就是说，Pell 方程

的全体解，恰好是

恰好是卢卡斯数列和斐波那契数列。因此有

矩阵形式

斐波那契数列的递推可以用矩阵乘法的形式表达：

设 $F_n = \begin{pmatrix} F_n & F_{n-1} \end{pmatrix}$ ，我们得到

于是我们可以用矩阵乘法在 $O(\log n)$ 的时间内计算斐波那契数列。此外，前一节讲述的公式也可通过矩阵对角化的技巧来得到。

快速倍增法

使用上面的方法我们可以得到以下等式：

于是可以通过这样的方法快速计算两个相邻的斐波那契数（常数比矩乘小）。代码如下，返回值是一个二元组。

```
1 pair<int, int> fib(int n) {
2   if (n == 0) return {0, 1};
3   auto p = fib(n >> 1);
4   int c = p.first * (2 * p.second - p.first);
5   int d = p.first * p.first + p.second * p.second;
6   if (n & 1)
7     return {d, c + d};
8   else
9     return {c, d};
10 }
```

性质

斐波那契数列拥有许多有趣的性质，这里列举出一部分简单的性质：

1. 卡西尼性质 (Cassini's identity)：。 $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$
2. 附加性质：。 $F_{n+1}F_{n-1} - F_n^2 = (-1)^n$
3. 取上一条性质中，我们得到。 $F_{n+1}F_{n-1} - F_n^2 = (-1)^n$
4. 由上一条性质可以归纳证明，。 $F_{n+1}F_{n-1} - F_n^2 = (-1)^n$
5. 上述性质可逆，即。 $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$
6. GCD 性质：。 $\gcd(F_n, F_m) = F_{\gcd(n, m)}$
7. 以斐波那契数列相邻两项作为输入会使欧几里德算法达到最坏复杂度（具体参见 [维基 - 拉梅](#)）。

斐波那契数列与卢卡斯数列的关系

不难发现，关于卢卡斯数列与斐波那契数列的等式，与三角函数公式具有很高的相似性。比如：

与

很像。以及

与

很像。因此，卢卡斯数列与余弦函数很像，而斐波那契数列与正弦函数很像。比如，根据

可以得到两下标之和的等式：

于是推论就有二倍下标的等式：

这也是一种快速倍增下标的办法。同样地，也可以仿照三角函数的公式，比如奇偶性、和差化积、积化和差、半角、万能代换等等，推理出更多有关卢卡斯数列与斐波那契数列的相应等式。

斐波那契编码

我们可以利用斐波那契数列为正整数编码。根据 [齐肯多夫定理](#)，任何自然数 可以被唯一地表示成一些斐波那契数的和：

并且（即不能使用两个相邻的斐波那契数）

于是我们可以用 的编码表示一个正整数，其中 则表示 被使用。编码末位我们强制给它加一个 1（这样会出现两个相邻的 1），表示这一串编码结束。举几个例子：

给 编码的过程可以使用贪心算法解决：

1. 从大到小枚举斐波那契数，直到 。
2. 把 减掉，在编码的 的位置上放一个 1（编码从左到右以 0 为起点）。
3. 如果 为正，回到步骤 1。
4. 最后在编码末位添加一个 1，表示编码的结束位置。

解码过程同理，先删掉末位的 1，对于编码为 1 的位置（编码从左到右以 0 为起点），累加一个 到答案。最后的答案就是原数字。

模意义下周期性

考虑模 意义下的斐波那契数列，可以容易地使用抽屉原理证明，该数列是有周期性的。考虑模意义下前 个斐波那契数对（两个相邻数配对）：

的剩余系大小为 m ，意味着在前 m 个数对中必有两个相同的数对，于是这两个数对可以往后生成相同的斐波那契数列，那么他们就是周期性的。

皮萨诺周期

模 m 意义下斐波那契数列的最小正周期被称为 [皮萨诺周期](#) (Pisano periods, [OEIS A001175](#))。

皮萨诺周期总是不超过 m ，且只有在满足 $m \equiv 0 \pmod{4}$ 的形式时才取到等号。

当需要计算第 n 项斐波那契数模 m 的值的时候，如果 n 非常大，就需要计算斐波那契数模 m 的周期。当然，只需要计算周期，不一定是最小正周期。

容易验证，斐波那契数模 m 的最小正周期是 $\pi(m)$ ，模 n 的最小正周期是 $\pi(n)$ 。

显然，如果 m 与 n 互素， $\pi(mn)$ 就是 $\pi(m)$ 与 $\pi(n)$ 的最小公倍数。

计算周期还需要以下结论：

结论 1：对于奇素数 p ， $\pi(p)$ 是斐波那契数模 p 的周期。即，奇素数 p 的皮萨诺周期整除 $p-1$ 。

证明：

此时 $p \nmid 2$ 。

由二项式展开：

因为 F_0 和 F_1 两项都同余于 0 ，与 F_2 和 F_3 一致，所以 $\pi(p)$ 是周期。

结论 2：对于奇素数 p ， $\pi(p^2)$ 是斐波那契数模 p^2 的周期。即，奇素数 p 的皮萨诺周期整除 p^2-1 。

证明：

此时 $p \nmid 2$ 。

由二项式展开：

模 p^2 之后，在 F_n 式中，只有 F_n 项留了下来；在 F_{n+1} 式中，有 F_n 、 F_{n-1} ，三项留了下来。

于是 F_n 和 F_{n+1} 两项与 F_0 和 F_1 一致，所以 $\pi(p^2)$ 是周期。

结论 3：对于素数 p ， $\pi(p^k)$ 是斐波那契数模 p^k 的周期，等价于 $\pi(p^{k-1})$ 是斐波那契数模 p^{k-1} 的周期。特别地， $\pi(p)$ 是模 p 的皮萨诺周期，等价于 $\pi(p)$ 是模 p 的皮萨诺周期。

证明：

这里的证明需要把 F_n 看作一个整体。

由于：

因此：

因为反方向也可以推导，所以 π 是斐波那契数模 m 的周期，等价于：

当 m 是奇素数时，由 [升幂引理](#)，有：

当 m 是偶数时，由 [升幂引理](#)，有：

代入 $x = F_{\pi}$ 和 $y = F_0$ ，为 $F_{\pi} \equiv F_0 \pmod{m}$ 和 $F_{\pi+1} \equiv F_1 \pmod{m}$ ，上述条件也就等价于：

因此也等价于 π 是斐波那契数模 m 的周期。

因为周期等价，所以最小正周期也等价。

三个结论证完。据此可以写出代码：

```

1 struct prime {
2     unsigned long long p;
3     int times;
4 };
5
6 struct prime pp[2048];
7 int pptop;
8
9 unsigned long long get_cycle_from_mod(
10     unsigned long long mod) // 这里求解的只是周期, 不一定是最小正周期
11 {
12     pptop = 0;
13     srand(time(nullptr));
14     while (n != 1) {
15         __int128_t factor = (__int128_t)100000000000 * 100000000000;
16         min_factor(mod, &factor); // 计算最小素因数
17         struct prime temp;
18         temp.p = factor;
19         for (temp.times = 0; mod % factor == 0; temp.times++) {
20             mod /= factor;
21         }
22         pp[pptop] = temp;
23         pptop++;
24     }
25     unsigned long long m = 1;
26     for (int i = 0; i < pptop; ++i) {
27         int g;
28         if (pp[i].p == 2) {
29             g = 3;
30         } else if (pp[i].p == 5) {
31             g = 20;
32         } else if (pp[i].p % 5 == 1 || pp[i].p % 5 == 4) {
33             g = pp[i].p - 1;
34         } else {
35             g = (pp[i].p + 1) << 1;
36         }
37         m = lcm(m, g * qpow(pp[i].p, pp[i].times - 1));
38     }
39     return m;
40 }

```

习题

- [SPOJ - Euclid Algorithm Revisited](#)
- [SPOJ - Fibonacci Sum](#)
- [HackerRank - Is Fibo](#)
- [Project Euler - Even Fibonacci numbers](#)
- [洛谷 P4000 斐波那契数列](#)

本页面主要译自博文 [Числа Фибоначчи](#) 与其英文翻译版 [Fibonacci Numbers](#)。其中俄文版版权协议为 **Public Domain + Leave a Link**；英文版版权协议为 **CC-BY-SA 4.0**。

本页面最近更新：2024/12/16 13:10:29, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[c-forrest](#), [Chrogeek](#), [Enter-tainer](#), [EntropyIncreaser](#), [FFjet](#), [Great-designer](#), [ImpleLee](#), [jifbt](#),

[Junyan721113](#), [ouuan](#), [sshwy](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用